SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1404 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I **A. Yuvan Charan (192424383)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **A. YUVAN CHARAN**

Grammar Lab: Intelligent LL (1) Parser Generator and Compiler Grammar Analysis Studio

DESIGN TASK

Submitted in the partial fulfilment for the award of the degree of

CSA1404 – Compiler Design

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted By

A Yuvan Charan (192424383)

Under the Supervision of

Dr. G. Anitha

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST - 2026

DESIGN TASK

QUESTION – 1

Grammar Validation and Recursive Descent Parsing

Aim

To design and implement a **Recursive Descent Parser** that validates a given Context-Free Grammar (CFG) and checks whether an input string belongs to the language generated by the grammar. The parser performs syntax analysis using a top-down parsing technique and displays whether the input string is accepted or rejected.

Introduction

A **compiler** is a software program that translates source code written in a high-level programming language into machine language or intermediate code. The compilation process consists of several phases, namely lexical analysis, syntax analysis, semantic analysis, code optimization, and code generation. Among these phases, **syntax analysis** plays an important role in verifying whether the source code follows the grammar rules of the programming language.

A **Recursive Descent Parser** is one of the simplest top-down parsing techniques used in compiler design. It consists of a collection of recursive procedures where each procedure corresponds to one non-terminal symbol in the grammar. Starting from the start symbol, the parser recursively expands productions and matches the input string with the grammar rules.

Recursive descent parsing works efficiently for grammars that do not contain left recursion and are suitable for predictive parsing. The parser reads one token at a time and decides which production rule to apply. If every symbol in the input string is successfully matched according to the grammar, the parser accepts the string; otherwise, it reports an error.

In the **GrammarLab** mini project, the Recursive Descent Parser has been integrated into an interactive web application developed using **Python Flask, HTML, CSS, and JavaScript**. The application validates the grammar, performs recursive parsing, displays parsing steps, and provides the final acceptance or rejection result through an easy-to-use graphical interface.

Grammar Used

$S \rightarrow aSb \mid \varepsilon$

Where

- **S** represents the Start Symbol.
- **a** and **b** are Terminal Symbols.
- **ε** represents the Empty String (Epsilon).

Sample Valid Strings

ab
aabb
aaabbb
aaaabbbb

Sample Invalid Strings

abb
aaabb
abab
aab

Working Principle of Recursive Descent Parser

The Recursive Descent Parser begins parsing from the **start symbol (S)** and recursively applies production rules until every input symbol is matched. During parsing, each grammar rule is represented by a separate recursive function. Whenever a terminal symbol is encountered, it is compared with the current input symbol. If both symbols match, the parser moves to the next input symbol; otherwise, parsing stops and an error is generated.

For the grammar

$S \rightarrow aSb \mid \varepsilon$

the parser performs the following operations:

1. Read the first input symbol.
2. If the symbol is **a**, match it.
3. Recursively call the function **S()**.
4. Match the corresponding **b**.
5. If no more symbols remain, apply the ε production.
6. Accept the string if the entire input has been successfully parsed.

This approach makes parsing simple, efficient, and easy to understand for educational purposes.

Algorithm

Algorithm: Recursive Descent Parsing

Step 1: Read the input string.
Step 2: Start parsing from the start symbol **S**.
Step 3: Compare the current input symbol with the grammar production.
Step 4: If the production contains a terminal symbol, match it with the input.
Step 5: If the production contains a non-terminal, recursively call its parsing function.
Step 6: Continue until all symbols are processed.
Step 7: If the input is completely matched, display **Accepted**.
Step 8: Otherwise display **Rejected**.

Pseudo Code

```
Function ParseS()

If current_symbol == 'a'

Match('a')

ParseS()

Match('b')

Else

Return

End Function
```

Flow of Parsing

Input String

↓

Read Grammar

↓

Start from S

↓

Match Terminal Symbols

↓

Expand Non-Terminals

↓

Input Completed?

↓

YES → ACCEPT

NO → REJECT

Execution of Recursive Descent Parser

Input Grammar

S → aSb | ε

Input String

aaabbb

Step-by-Step Parsing

Step	Current Symbol	Action
1	**a**	**Match 'a'**
2	**a**	**Match 'a'**
3	**a**	**Match 'a'**
4	**ε**	**Apply ε Production**
5	**b**	**Match 'b'**
6	**b**	**Match 'b'**
7	**b**	**Match 'b'**
8	**End**	**Accept Input**

Explanation

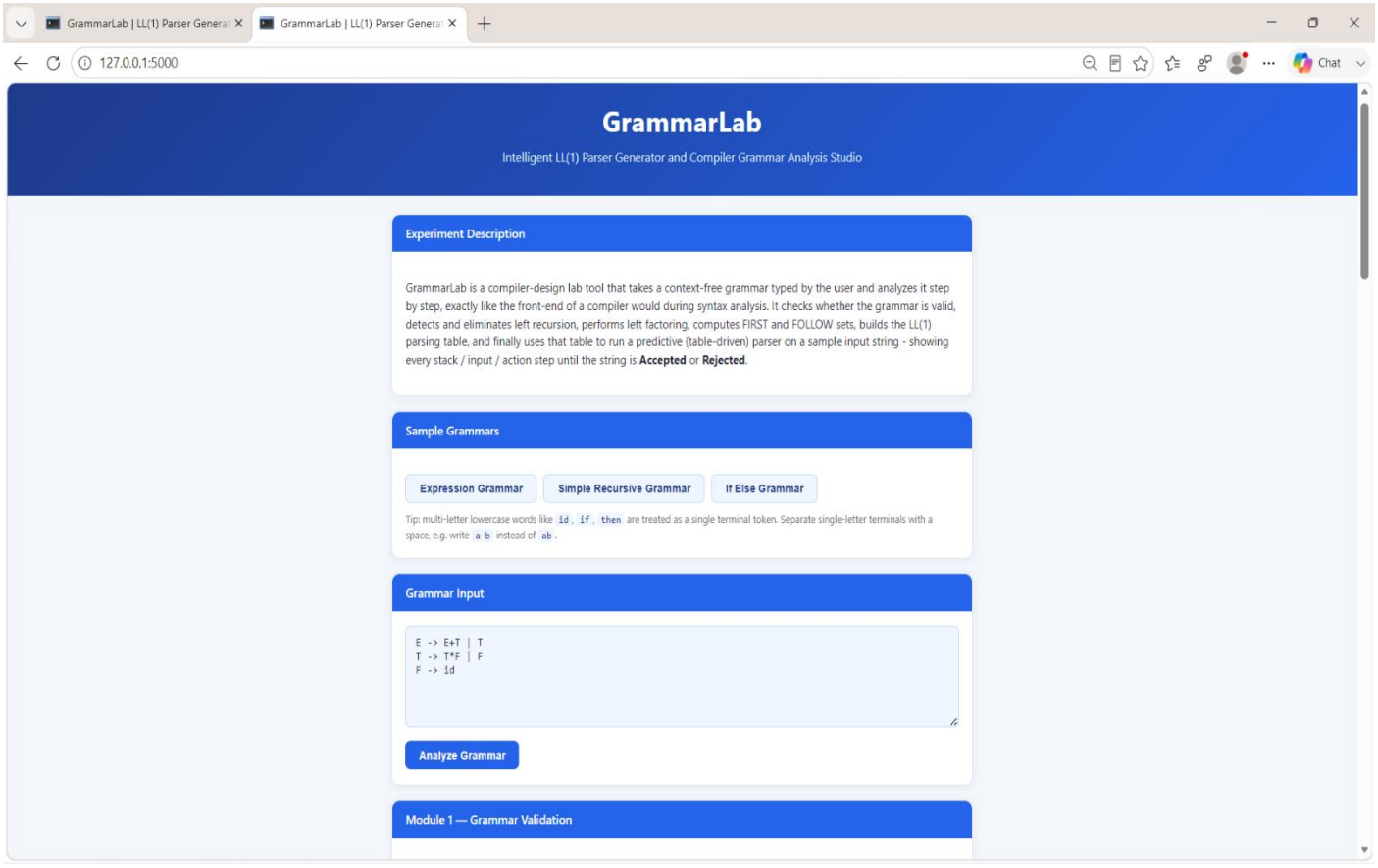
The parser starts from the non-terminal **S** and applies the production rule **S → aSb** repeatedly while reading the symbol **a**. Once all opening symbols have been matched, the parser applies the ε production and begins matching the closing symbols **b** in reverse order. Since every symbol in the input string satisfies the grammar, the parser successfully reaches the end of the input and accepts the string.

This demonstrates the working of a Recursive Descent Parser using recursive function calls. The parser validates the syntax by following the grammar rules exactly as defined.

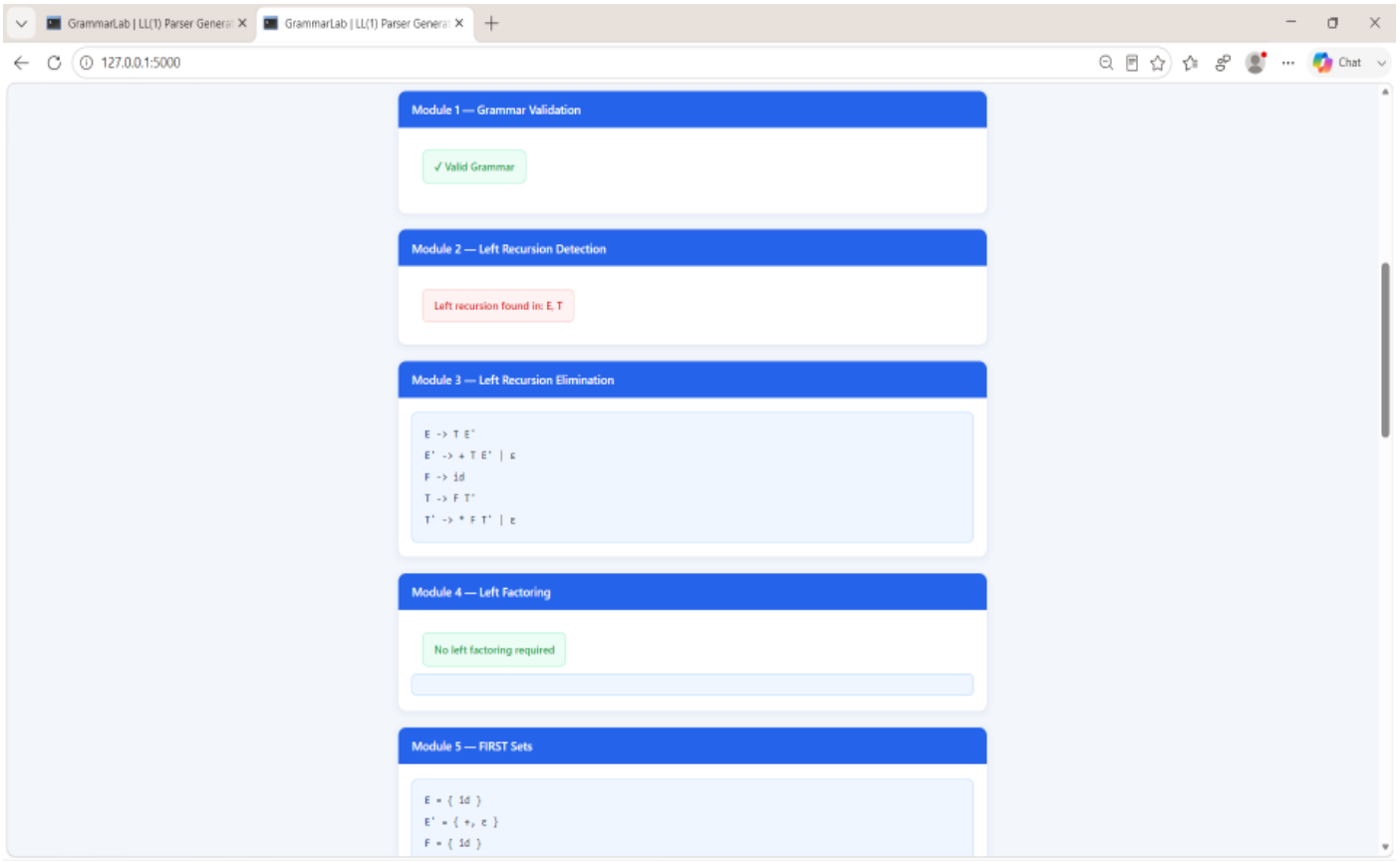
Advantages

- Simple to understand and implement.
- Suitable for educational compiler projects.
- Efficient for LL(1) grammars.
- Provides clear parsing steps.
- Easy to debug and maintain.

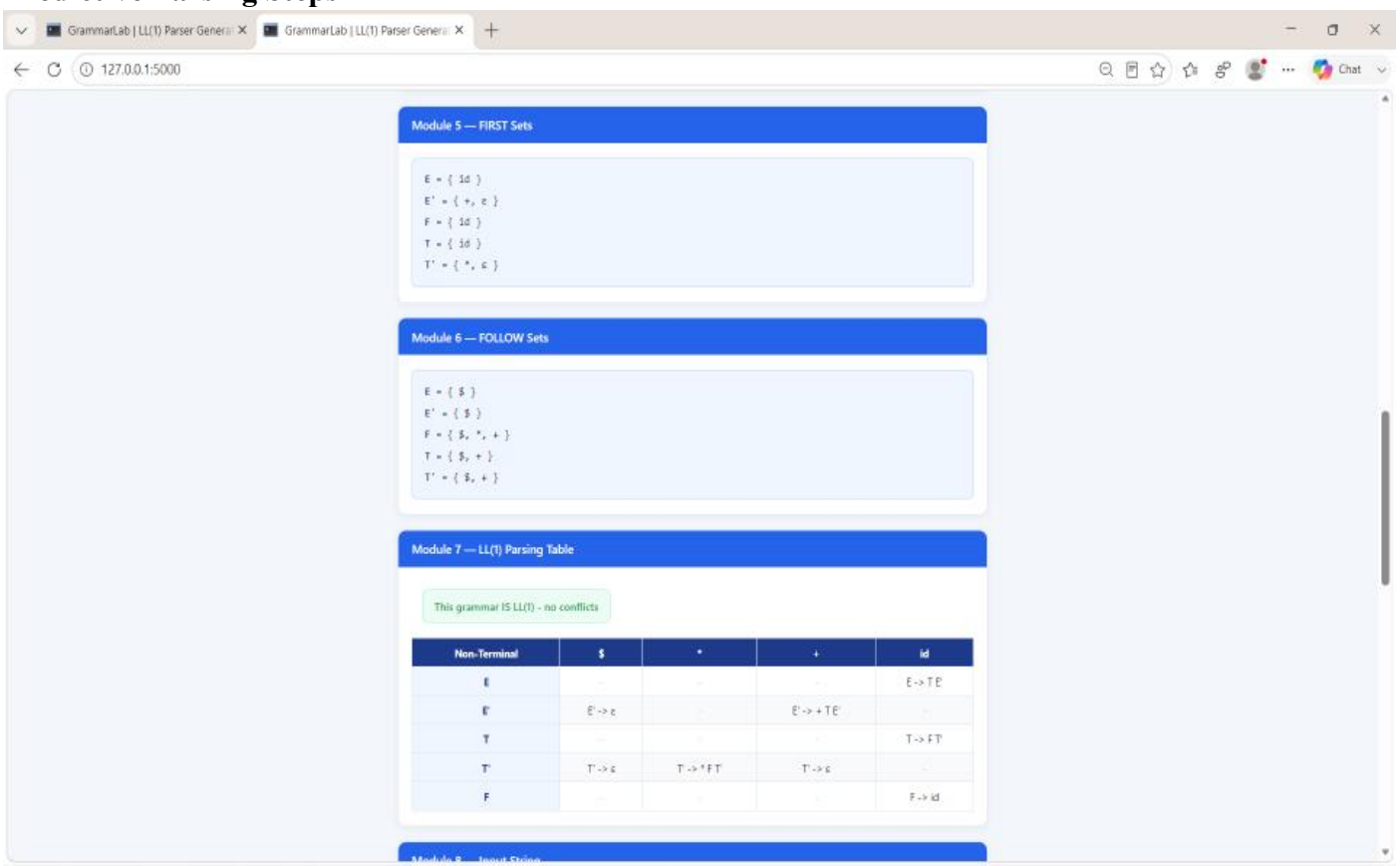
Output and Result



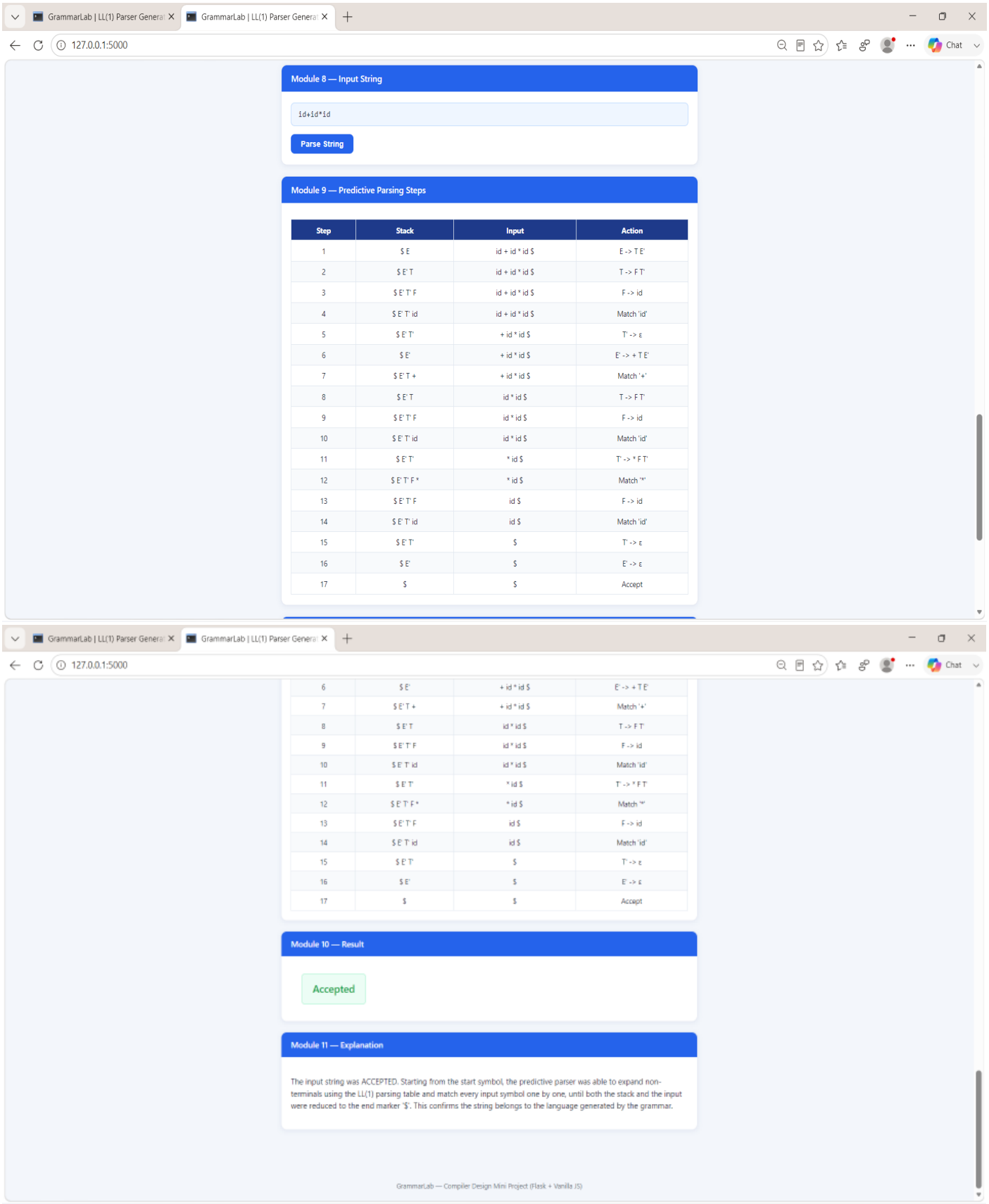
Grammar Validation



Predictive Parsing Steps



Accepted Result



Result

The Recursive Descent Parser module was successfully implemented in the **GrammarLab** mini project using **Python Flask, HTML, CSS, and JavaScript**. The application correctly validates the given grammar, performs recursive parsing, displays each parsing step, and determines whether the input string belongs to the language generated by the grammar. The parser successfully accepted valid input strings and rejected invalid strings, demonstrating the correctness of the implemented parsing algorithm.

QUESTION – 2

Left Recursion Detection and Elimination

Aim

To identify left recursion in a Context-Free Grammar (CFG) and eliminate it by transforming the grammar into an equivalent form suitable for **LL(1) predictive parsing**. This process ensures that the grammar can be parsed efficiently using top-down parsing techniques without causing infinite recursion.

Introduction

Left recursion is one of the most common problems encountered in compiler design while constructing top-down parsers. A grammar is said to be **left recursive** if a non-terminal appears as the leftmost symbol on the right-hand side of one of its own productions. Such grammars cannot be directly used by Recursive Descent or LL(1) parsers because they lead to infinite recursive function calls.

For example, the production:

$E \rightarrow E + T$

is left recursive because the non-terminal **E** appears first on the right-hand side. When a Recursive Descent Parser attempts to parse this production, it repeatedly calls the function corresponding to **E** without consuming any input symbols, resulting in an infinite loop.

To overcome this issue, the grammar must be transformed into an equivalent grammar that removes left recursion while preserving the language generated by the original grammar. This transformed grammar becomes compatible with LL(1) parsing techniques and allows efficient syntax analysis.

The **GrammarLab** mini project automatically detects left recursion, identifies the affected non-terminals, and generates a transformed grammar that can be directly used for predictive parsing.

Original Grammar

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

Why Left Recursion Must Be Eliminated

- Prevents infinite recursive function calls.
- Makes the grammar compatible with LL(1) parsers.
- Improves parsing efficiency.
- Simplifies parser implementation.
- Enables predictive parsing.

Expected Output

The application should detect that **E** and **T** contain left recursion and display an appropriate message before generating the transformed grammar.

Left Recursion Elimination

After detecting left recursion, the grammar is transformed into an equivalent grammar that removes recursive productions. A new non-terminal is introduced to preserve the language while avoiding infinite recursion.

Transformation Rule
If a production is of the form:
$A \rightarrow A\alpha \mid \beta$
It is transformed into:
$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \varepsilon$
Where:

- **A** is the non-terminal.
- **α** represents the recursive part.
- **β** represents the non-recursive production.
- **ε** represents the empty string.

Transformed Grammar
$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *\ F\ T' \mid \varepsilon$

$F \rightarrow id$

Algorithm
Algorithm for Left Recursion Elimination
Step 1: Read all grammar productions.
Step 2: Check whether the leftmost symbol of each production is the same as the left-hand side non-terminal.
Step 3: If left recursion exists, separate recursive and non-recursive productions.
Step 4: Create a new non-terminal using the apostrophe (') notation.
Step 5: Rewrite the grammar using the transformation rule.
Step 6: Display the transformed grammar.

Advantages of Left Recursion Elimination

- Removes infinite recursion.
- Produces an LL(1) compatible grammar.
- Improves parser performance.
- Makes syntax analysis easier.
- Simplifies grammar processing.

Explanation
The GrammarLab application successfully analyzed the input grammar and identified left-recursive productions in the non-terminals **E** and **T**. After detection,

Result
The Left Recursion Detection and Elimination module was successfully implemented in the **GrammarLab** mini project using **Python Flask, HTML, CSS, and JavaScript**.

QUESTION – 3

FIRST Set, FOLLOW Set and LL(1) Parsing Table

Aim

To compute the **FIRST** and **FOLLOW** sets for a given Context-Free Grammar (CFG) and construct the **LL(1) Parsing Table** for predictive parsing. The generated parsing table is used to determine whether the grammar is suitable for LL(1) parsing and to guide the syntax analysis process.

Introduction

In Compiler Design, **FIRST** and **FOLLOW** sets are essential components used in the construction of an **LL(1) Parsing Table**. These sets help the parser determine which production rule should be selected during syntax analysis without backtracking.

The **FIRST Set** of a grammar symbol contains all the terminal symbols that can appear as the first symbol of strings derived from that symbol. If a non-terminal can derive an empty string (ε), then ε is also included in its FIRST set.

The **FOLLOW Set** of a non-terminal contains all terminal symbols that can immediately appear to the right of that non-terminal in some valid derivation. For the start symbol, the end-of-input marker **($)** is always included in its FOLLOW set.

Once the FIRST and FOLLOW sets are generated, they are used to build the **LL(1) Parsing Table**. This table enables the predictive parser to select the correct production based on the current input symbol and the top of the parsing stack. If each table entry contains only one production, the grammar is considered **LL(1)** and is suitable for predictive parsing.

In the **GrammarLab** application, the computation of FIRST sets, FOLLOW sets, and LL(1) parsing tables is performed automatically, providing students with a clear understanding of the grammar analysis process.

Grammar Used

E → T E'

E' → + T E' | ε

T → F T'

T' → * F T' | ε

F → id

Objectives

- Compute FIRST sets for all non-terminals.
- Compute FOLLOW sets for all non-terminals.
- Verify whether the grammar is LL(1).
- Construct the LL(1) Parsing Table.
- Display the results in a user-friendly format.

Generation of FIRST, FOLLOW and LL(1) Parsing Table

FIRST Sets

The FIRST set contains the first terminal that can appear in the derivation of a grammar symbol.

Generated FIRST Sets

FIRST(E) = { id }

FIRST(E') = { + , ε }

FIRST(T) = { id }

FIRST(T') = { * , ε }

FIRST(F) = { id }

FOLLOW Sets

The FOLLOW set contains all terminal symbols that can appear immediately after a non-terminal during parsing.

Generated FOLLOW Sets

FOLLOW(E) = { $ }

FOLLOW(E') = { $ }

FOLLOW(T) = { + , $ }

FOLLOW(T') = { + , $ }

FOLLOW(F) = { * , + , $ }

Construction of LL(1) Parsing Table

The LL(1) Parsing Table is constructed using the computed FIRST and FOLLOW sets. Each row represents a non-terminal, while each column represents an input terminal. The table guides the predictive parser in selecting the correct production rule.

Rules for Constructing the Parsing Table

1. For every production **A → α**, place the production in all table entries corresponding to **FIRST(α)**.
2. If **ε** belongs to **FIRST(α)**, place the production in all entries corresponding to **FOLLOW(A)**.
3. If no conflicts occur, the grammar is considered an LL(1) grammar.

The GrammarLab application automatically performs these steps and generates the parsing table in tabular format.

Advantages

- Eliminates parsing ambiguity.
- Enables predictive parsing without backtracking.
- Simplifies parser implementation.
- Detects grammar conflicts.
- Improves syntax analysis efficiency.

Explanation

The GrammarLab application successfully generated the FIRST and FOLLOW sets for all non-terminals in the transformed grammar. These sets were then used to construct the LL(1) Parsing Table. Since each table entry contains only one production rule and no conflicts were detected, the grammar satisfies the requirements of an LL(1) grammar.

Result

The FIRST Set, FOLLOW Set, and LL(1) Parsing Table modules were successfully implemented in the **GrammarLab: Intelligent LL(1) Parser Generator and Compiler Grammar Analysis Studio** using **Python Flask, HTML, CSS, and JavaScript**.

QUESTION – 4

Predictive Parsing and Grammar Analysis

Aim

To implement a **Predictive Parser** using the generated LL(1) Parsing Table and analyze whether the given input string conforms to the grammar rules. The parser should display the parsing steps, validate the input string, and produce the final result as **Accepted** or **Rejected**.

Introduction

Predictive Parsing is a **top-down parsing technique** used in Compiler Design for syntax analysis. It predicts which production rule should be applied by using a single look-ahead input symbol and an LL(1) Parsing Table. Since the parser does not require backtracking, predictive parsing is faster and more efficient than general recursive parsing techniques.

The Predictive Parser maintains two important components:

- **Parsing Stack**
- **Input Buffer**

Initially, the stack contains the start symbol and the end marker **($)**, while the input buffer contains the input string followed by **($)**. At each step, the parser compares the top of the stack with the current input symbol. If they match, the symbol is removed from the stack and the parser proceeds to the next input symbol. Otherwise, the LL(1) Parsing Table is consulted to determine the appropriate production rule.

In the **GrammarLab** mini project, predictive parsing has been implemented using the generated LL(1) Parsing Table. The application displays every parsing step, making it easy for students to understand how syntax analysis works internally.

Input Grammar

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *\ F\ T' \mid \varepsilon$

$F \rightarrow id$

Input String

id + id * id

Objectives

- Accept grammar from the user.
- Parse the input using the LL(1) Parsing Table.
- Display stack operations.
- Show parser actions.
- Accept or reject the input string.

Execution of Predictive Parsing

The Predictive Parser processes the input string by repeatedly comparing the stack with the current input symbol. If the top of the stack is a non-terminal, the parser selects the corresponding production rule from the LL(1) Parsing Table. If the top of the stack is a terminal and it matches the input symbol, both are removed. The process continues until both the stack and input contain only the end marker ($).

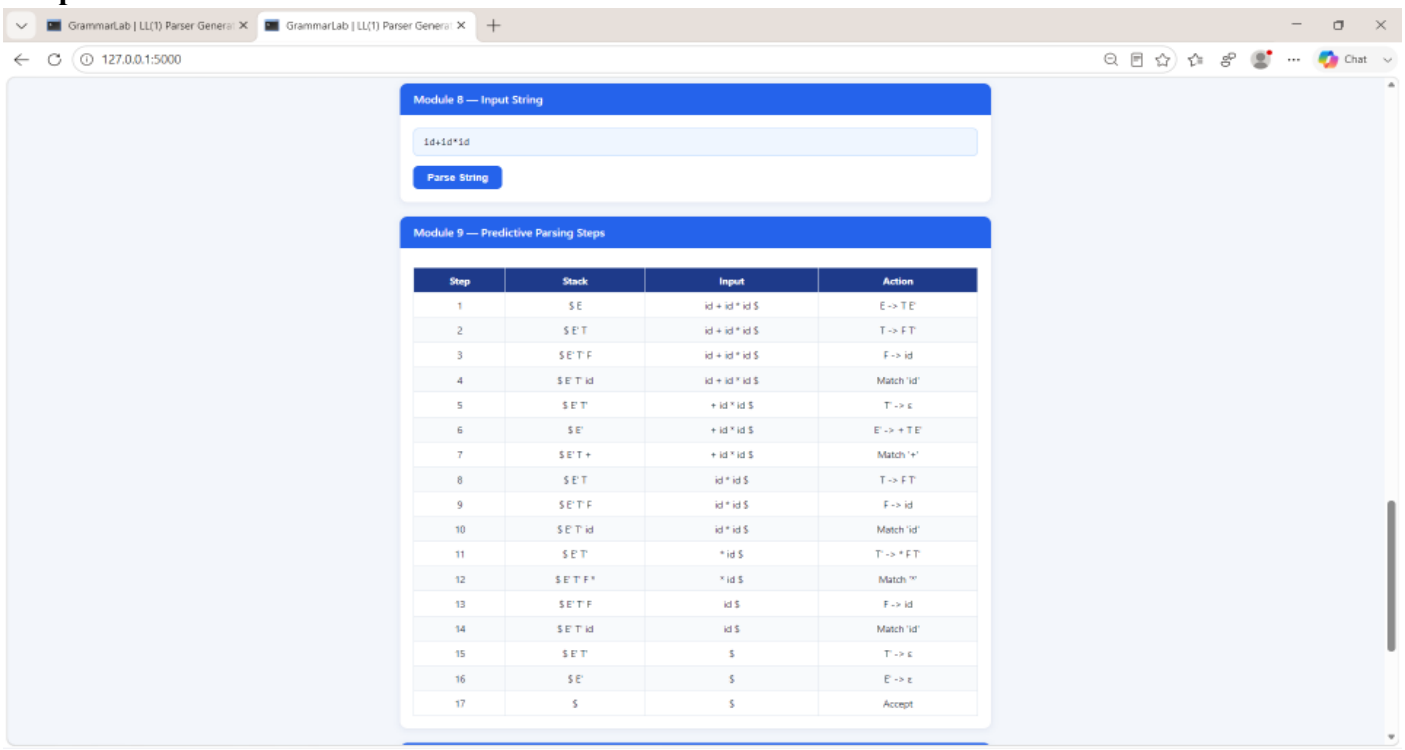
Sample Parsing Steps

Step	Stack	Input	Action
1	E $	id + id * id $	Apply E → T E'
2	T E' $	id + id * id $	Apply T → F T'
3	F T' E' $	id + id * id $	Apply F → id
4	id T' E' $	id + id * id $	Match id
5	T' E' $	+ id * id $	Apply T' → ε
6	E' $	+ id * id $	Apply E' → + T E'
7	+ T E' $	+ id * id $	Match +
8	T E' $	id * id $	Apply T → F T'
9	F T' E' $	id * id $	Apply F → id
10	id T' E' $	id * id $	Match id
11	T' E' $	* id $	Apply T' → * F T'
12	* F T' E' $	* id $	Match *
13	F T' E' $	id $	Apply F → id
14	id T' E' $	id $	Match id
15	T' E' $	$	Apply T' → ε
16	E' $	$	Apply E' → ε
17	$	$	**Accept**

Explanation

The parser successfully matched every symbol in the input string according to the grammar rules. Since both the stack and input reached the end marker simultaneously, the parser accepted the string. This confirms that the given input follows the grammar and is syntactically correct.

Output Screenshot



Features of GrammarLab

The developed GrammarLab application integrates multiple compiler design concepts into a single user-friendly platform. The major features of the application include:

- Grammar Validation
- Left Recursion Detection
- Left Recursion Elimination
- Left Factoring
- FIRST Set Generation
- FOLLOW Set Generation
- LL(1) Parsing Table Generation
- Predictive Parsing
- Input String Validation
- Step-by-Step Parsing Process
- Accepted/Rejected Result Display
- Interactive Web Interface
- Fast and Accurate Grammar Analysis

These features make GrammarLab an effective educational tool for learning syntax analysis and predictive parsing.

Conclusion

The **GrammarLab: Intelligent LL(1) Parser Generator and Compiler Grammar Analysis Studio** was successfully designed and implemented as a Compiler Design mini project using **Python Flask, HTML, CSS, and JavaScript**. The application combines multiple compiler concepts into a single interactive platform, allowing users to analyze context-free grammars efficiently.

The project begins by validating the grammar and checking its correctness. It then detects and eliminates left recursion, performs left factoring when required, and computes the FIRST and FOLLOW sets for all non-terminals. Using these sets, the application automatically constructs the LL(1) Parsing Table, which serves as the basis for predictive parsing.

The Predictive Parser analyzes the input string step by step, displaying stack operations, parser actions, and the final parsing result. This visual representation helps users understand how syntax analysis is performed inside a compiler. The application also provides clear explanations for accepted or rejected strings, making it suitable for both learning and demonstration purposes.

One of the major strengths of GrammarLab is its simple and intuitive web interface. Unlike traditional command-line programs, the application allows users to perform grammar analysis through an interactive graphical environment. This improves usability and enhances the learning experience for students studying Compiler Design.

Overall, the project successfully demonstrates the implementation of key concepts such as grammar validation, left recursion elimination, FIRST and FOLLOW set computation, LL(1) parsing table construction, and predictive parsing. The developed system meets the objectives of the assessment and provides a practical understanding of syntax analysis techniques used in modern compiler construction.

Future Enhancements

- Support for LR(0), SLR(1), CLR(1), and LALR(1) parsers.
- Parse Tree visualization using graphical diagrams.
- Syntax Tree generation.
- Semantic Analysis module.
- Intermediate Code Generation.
- Grammar conflict detection with suggestions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	10	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation